EASILE 2D 게임 프로그램 설계서

v0.15 · Baseline 1.0 및 PATCH-01 방어 코드 반영 · 2026-10-03

대상: Windows x64 단일 프로세스 GameClient.exe. 이 문서는 게임 전체의 현행 구조와 추가된 방어 경로를 설명한다. Baseline 승인 사실과 변경된 작업 트리의 검증 상태는 구분한다.

1. 문서 목적과 기준

EASILE은 자체 제작 2D 탑다운 미니 MOBA를 보안 실험 대상으로 사용하는 프로젝트다. 본 문서는 v0.14 설계서의 게임 기능을 유지하면서 이후 소스에 추가된 IntegrityGuard와 런타임 쿨다운 검사를 전체 프로그램 아키텍처에 통합한 개정본이다. 코드에서 확인되지 않은 방어 효과를 완료 결과로 쓰지 않는다.

Baseline 1.0은 2026-09-30 사용자가 직접 플레이하고 승인한 게임 상태다. 당시 Debug/Release 빌드와 각 49개 자동 테스트가 통과했다. 현재 작업 트리에는 그 이후의 변경이 있으므로 이 문서의 '현재 구현'은 Baseline 바이너리와 동일하다는 뜻이 아니다. 공격 전·후·방어 빌드는 해시로 따로 식별한다.

2. 실행 환경과 배포 단위

솔루션 EASILE.sln은 GameClient와 GameTests 프로젝트로 구성된다. 플랫폼은 Windows x64, Visual Studio 2022/MSVC v143, C++17, Windows SDK, DirectX 11이다. GameClient는 Win32 창과 DX11 월드 화면, GDI 자식 HUD를 사용하는 단일 프로세스 실행 파일이다. GameTests는 월드 규칙과 회귀를 검사하는 별도 실행 파일이다.

Application은 창 생성, Renderer/HUDRenderer 초기화, 메시지 처리, 입력 읽기, 고정 10 ms 시뮬레이션 및 프레임 출력을 담당한다. GameTime은 steady_clock으로 프레임 델타를 만든다. 실행 옵션 --smoke-gui는 창과 렌더 경로를 몇 프레임 실행하는 검증 경로다. Release 최적화는 MaxSpeed이고 프로젝트 파일에서 WholeProgramOptimization은 false다. 빌드 보안 옵션은 실제 생성된 바이너리와 프로젝트 설정을 함께 확인해야 한다.

3. 프로그램 전체 구성

3.1 계층과 데이터 흐름

입력 경로: Win32 메시지·키보드·마우스 → InputManager::Read → PlayerCommand → Application의 10 ms 입력 보존 → PlayerController::BuildCommand → GameWorld::Update. 화면 경로: GameWorld::CreateSnapshot 및 CreateHud → Renderer(DX11)와 HUDRenderer(GDI). 기록 경로: GameWorld::Log → Logger → GameClient.log.

게임 규칙은 GameWorld가 조정한다. Hero는 체력·성장·쿨다운·사망·부활을 보유하고 PlayerHero/BotHero가 공통 Hero를 상속한다. 세 Bot은 BotController의 판단과 각각 독립 BotRuntime을 사용한다. CombatSystem은 피해와 사망을 판정한다. GameWorld는 Minion·Tower·Base·Projectile·Brush·회복 픽업과 경기 결과를 소유하고 갱신한다. MapPaths는 도로·정글·벽·경로·이동 및 공격 차단의 공통 기하 규칙을 제공한다.

보안 경로: GameWorld::Update의 Player 궁극기 분기 → IntegrityGuard::VerifyUpdateIntegrity → simulationTime_ 기반 10초 검사 → Hero::UltimateReady → Projectile 생성 및 Hero::StartUltimate → lastUltimateUseTime_ 갱신. 검사가 실패하면 SECURITY 로그를 남기고 해당 Update 호출을 반환한다. 보안 검사는 월드 갱신 경로에 있으므로 정상 입력·시간·전투 규칙과 함께 설계·시험해야 한다.

3.2 모듈 책임과 경계

InputManager는 입력 상태와 새 누름 edge를 PlayerCommand로 변환하며 HP·XP를 수정하지 않는다. PlayerController는 사망 중 이동·공격을 막고 이동 벡터를 정규화한다. GameWorld는 update 순서와 객체 소유권, Bot 동작, 표적 선택, 발사, 충돌, 경험치 귀속, 승패를 결정한다. CombatSystem은 Hero/Minion/Tower/Base에 대한 피해 계산과 사망 여부를 반환한다. VisibilitySystem은 수풀·revealed 상태를 판정한다.

Snapshot의 EntitySnapshot은 종류·위치·크기·팀·visible·alive·HP 비율·ID 등의 읽기 전용 값을 담는다. Renderer는 visible && alive 객체만 그리며 카메라 오프셋을 적용한다. HUDViewModel은 HP, 레벨, XP, 공격 피해, 궁극기 남은 시간, 조준 방식, 표적, 결과를 담고 HUDRenderer가 읽기만 한다. 숨은 객체가 Snapshot에는 남는 구조는 렌더 변조 실험의 관찰 지점이기도 하다.

IntegrityGuard는 EXE 메모리의 지정 바이트를 CRC32로 비교하는 독립 네임스페이스다. Logger는 벽시각과 시뮬레이션 시간, category/event/entity/detail을 텍스트로 기록한다. 현재 Logger가 실험 ID·빌드 해시를 자동 기입하는 구조는 확인되지 않으므로 실험 메타데이터는 별도 결과표에서 연결해야 한다.

4. 게임 상태와 수명주기

GameState는 Menu, Initializing, Playing, Paused, Result다. Enter 입력으로 Restart/Initialize를 수행하고 Playing으로 전환한다. P는 Playing과 Paused를 전환한다. Result에서는 R로 새 월드를 만든다. Playing이 아니거나 deltaSeconds가 0 이하일 때 전투·타이머 update는 진행하지 않는다. Result는 Nexus 파괴 시 Victory/Defeat/Draw를 표시한다.

Initialize는 Player 1명, Blue Ally Bot 1명, Red Bot 2명과 BotRuntime 3개를 초기화하고 양 팀의 Tower·Nexus·수풀·최초 Minion wave를 설정한다. simulationTime_, waveTimer_, pickupTimer_, lastUltimateUseTime_, 조준 방향, 투사체와 결과를 재설정한다. 마지막 궁극기 시각은 -10초로 시작해 첫 발사를 허용한다.

HeroState는 Alive/Dead다. 사망 후 대기 시간은 레벨에 따라 5~11초이며 자기 진영의 Spawn 위치에서 최대 HP로 부활한다. BotState는 Patrol/Push/Engage/Retreat/Recover/Defend/Dead다. BotRuntime은 상태 타이머, 공격 주기, 표적 ID, Lane, 후퇴 지점, 회복 및 픽업 추적값을 Bot마다 별도로 보유한다.

5. 맵, 이동과 시야

월드 크기는 1920×1440, 플레이어 추적 화면은 1280×720이다. 상·중·하 Lane 중심 y는 360/720/1080이고 도로 폭은 280이다. 각 팀의 Base는 x=105/1815, Tower는 x=390/1530에 Lane별로 배치된다. 중앙 정글은 세 Lane을 잇는 통로와 두 공간, 벽 4개 및 회복 픽업을 가진다. Brush는 6개다.

MapPaths::Contains/MoveInside가 Hero와 Minion의 이동 가능한 지형을 제한한다. 벽은 몸체 이동과 투사체 경로를 막는다. AttackBlocked는 벽뿐 아니라 서로 다른 Lane 사이의 직접 공격을 제한한다. 카메라 오프셋은 화면 좌표와 마우스 월드 좌표 변환에 적용한다. HUD는 월드 카메라와 독립적으로 화면에 고정된다.

수풀 안 Hero는 같은 수풀에 있는 관찰자에게 보이거나 공격 후 1.5초 reveal 상태일 때 표시된다. 사망 Hero는 렌더와 자동 표적에서 제외한다. VisibilitySystem의 판정은 Player 시야와 Bot 표적 선택에 사용되며, Minion·Tower에는 Hero 수풀 은폐를 적용하지 않는다.

6. 입력, 전투와 성장

Enter는 시작, WASD/방향키는 정규화된 이동, P는 일시정지·재개, R은 결과 화면 재시작이다. LMB는 클릭한 월드 좌표로 Basic을 쏘고 K는 Auto/Mouse 모드에 따른 Basic을 쏜다. J는 Ultimate를 요청한다. Tab은 K/J의 조준 모드를 전환한다. 1/2/3은 이후 레벨업의 HP/Attack/MoveSpeed 선호를 정한다. 기본 선호는 Attack이다. LMB·K·J는 새 누름 edge만 처리하며 키를 계속 누른다고 반복 발사하지 않는다.

Basic 피해는 기본 5, Ultimate 피해는 기본 30이다. K 자동 조준 사거리는 700, J는 1000이며 보이고 공격 가능한 적을 대상으로 한다. 플레이어 Basic에는 별도 재사용 대기 시간이 없고 Ultimate 기본 쿨다운은 10초다. Bot Basic은 별도의 AI 공격 주기를 사용한다. GameWorld가 Projectile을 소유하고 거리·충돌·벽 판정을 거쳐 피해를 적용한다. 발사자 ID는 XP 귀속에 사용된다.

12초마다 팀·Lane별 Minion 2기씩 생성된다. Minion은 자기 Lane의 살아 있는 적 Tower, 적 Minion, 적 Nexus 순으로 공격 대상을 판단한다. Tower는 사거리 내 적 Minion 또는 Hero를 공격한다. Nexus가 파괴되면 승패를 결정하고 두 진영의 파괴 시뮬레이션 시각 차이가 10 ms 이내면 Draw다.

Hero는 Minion/적 Hero 처치 경험치로 최대 7레벨까지 즉시 성장한다. 필요 XP는 100→120→144→173→208→250이고 초과 XP를 보존한다. 강화는 HP +20, Basic +2 및 Ultimate +8,

이동속도 +15 중 사전 선호를 적용한다. 마지막 피해 후 5초가 지난 생존 Hero는 초당 2 HP를 회복한다. Bot의 후퇴·회복 상태에는 별도 중첩 제한이 있다. 정글 픽업은 시작 20초 뒤 등장하고 20 HP를 회복하며 획득 후 30초 뒤 재생성된다.

7. 프레임 업데이트 순서와 시간 기준

Application은 프레임 시간을 최대 0.25초까지 누적하고 0.01초 단위로 GameWorld::Update를 호출한다. edge 입력은 첫 스텝에서만 사용되도록 보존·소비한다. 포커스를 잃으면 gameplay 입력을 버린다. Pause/Result에서는 누적 시간을 지우고 0초 update로 상태 명령만 처리한다.

Playing 스텝에서 GameWorld는 입력 명령을 정리하고 simulationTime_을 증가시킨 뒤 Hero 타이머·부활·자연 회복·이동·조준을 갱신한다. Player Basic과 Ultimate 요청을 처리한 다음 Minion wave, Bot, Projectile, Minion, Tower, 회복 픽업, 결과 판정 순으로 진행한다. 시간 기반 기능은 시뮬레이션 스텝에서만 갱신된다. 따라서 Pause 중 궁극기 쿨다운과 방어용 경과 시간도 멈춘다.

8. 추가된 방어 설계

8.1 코드 무결성 검사

IntegrityGuard::VerifyUpdateIntegrity는 GetModuleHandleW(nullptr)로 현재 EXE base를 구하고, 고정 UPDATE_RVA 0x14480부터 UPDATE_SIZE 0x13B4바이트를 읽어 CRC32를 계산한다. 계산값은 EXPECTED_UPDATE_CRC 0x37857DAE와 비교한다. 계산값은 OutputDebugStringA로 출력된다. GameWorld::Update의 J 분기에서 false이면 SECURITY/INTEGRITY_FAIL을 기록하고 반환한다.

이 구현은 특정 빌드·메모리 구간에 종속된다. RVA, 검사 길이, 기대 CRC가 생성된 EXE의 실제 GameWorld::Update 코드와 일치하는지 빌드마다 확인해야 한다. 현재 소스에는 PE 섹션 경계 검사나 기대값 자동 생성 절차가 없다. CRC32는 암호학적 인증이 아니며 검사 코드·기대값이 같은 프로세스에 있으므로 우회 가능성을 전제로 평가한다.

8.2 궁극기 경과 시간 검사

GameWorld는 lastUltimateUseTime_을 별도로 유지한다. J 입력 때 simulationTime_ − lastUltimateUseTime_가 10초 미만이면 SECURITY/RUNTIME_COOLDOWN_FAIL을 기록하고 해당 Update를 반환한다. 경과 시간이 충분하고 Hero::UltimateReady()도 true일 때만 투사체 생성·StartUltimate·마지막 성공 시각 갱신이 이루어진다. 즉 Hero의 ultimateCd_ 상태와 마지막 성공 시각을 모두 검사한다.

이 검사는 Player 궁극기 입력 경로에 있다. Bot 궁극기, 일반 공격, 프로세스 시작, 주기 검사에 자동 적용되지 않는다. 실패 시 게임 프로세스를 종료하거나 ultimateCd_를 복구하지 않는다. 방어 시각은

같은 GameWorld의 simulationTime_이며 외부 독립 시계가 아니다. 이 범위를 넘어선 탐지·차단 효과는 시험 후에만 주장한다.

8.3 공격 기록과 설계의 관계

PATCH-01 보고서는 Ultimate 10초→1초를 제목으로 사용하지만 변경 바이트와 결과 표는 10초→0초를 기록한다. 기록된 공격 결과는 0초 패치 관찰로 분류하고, 1초 패치는 별도 변형 시험으로 둔다. 보고서의 '종료·쿨다운 값 복구'는 방어 의도이나 현재 코드와 다르다. 방어 구현·정상 빌드·동일 공격 재시험 결과 칸은 완결된 증거로 볼 수 없다.

9. 요구사항과 검증 연결

게임 요구사항 G01~G04: 시작·재시작, 이동·조준·Basic/Ultimate, Bot/Minion/Tower/Nexus 전투, Victory/Defeat/Draw. G05~G08: XP·레벨업·부활·회복, Brush 시야, Pause 시간 정지, DX11/HUD/카메라. 해당 기능은 GameWorld·Hero·MapPaths·Renderer 및 GameTests의 G01~G28 회귀와 v0.14 검증 기록에 연결된다.

보안 요구사항 S01: 식별된 Release 빌드의 무결성 검사 구간과 기대 CRC를 확인한다. S02: 정상 J 입력에서 오탐 없이 10초 규칙을 유지한다. S03: 10초 미만 재요청 및 0초/1초 패치 변형에서 SECURITY 로그와 실제 Projectile 미생성을 함께 관찰한다. S04: Pause·Restart·사망·부활과 함께 동작한다. S05: 방어 전후 성능·회귀·재공격 결과를 같은 절차와 해시로 비교한다. 현재 테스트 소스에는 IntegrityGuard와 SECURITY 이벤트를 직접 검증하는 시험이 확인되지 않아 S01~S05의 실행 판정은 미확정이다.

빌드 검증은 Debug/Release x64, GameTests, --smoke-gui, 수동 조작을 구분해 기록한다. Baseline 시점의 49 PASS와 사용자 직접 플레이 승인은 현재 방어 후보 빌드의 통과 증거가 아니다. 재검증에서 실패한 경우 구현 또는 검사 상수의 문제를 분석하고 결과표를 갱신한다.

10. 실험 운영과 변경 관리

원본 Baseline EXE, 공격 패치 EXE, 방어 후보 EXE를 각기 별도 보존한다. 각 파일의 SHA-256, Git commit 또는 미커밋 변경 목록, 빌드 구성, 컴파일러 옵션, OS/VM, 입력 절차와 시뮬레이션 시각을 기록한다. Baseline 확정 문서의 Release 해시 4AC67F48BE58140159375027C69BCF65A22696E1C62CDD04CDEE38BD5AEC5DC9는 당시 실행 파일을 식별하며 현재 재빌드 산출물의 값으로 전용하지 않는다.

A1 Patch 다음 A2 런타임 값 변조, A3 함수 호출 경로 Hooking, A4 DX11 렌더 경로 Hooking은 후속 실험 계획이다. 각 단계의 방어는 구현·정상 검증·동일 공격 재시험·우회 변형을 끝낸 뒤 완료로 표시한다. 공격 실험은 본인 소유 프로그램과 격리 환경에서만 수행한다.